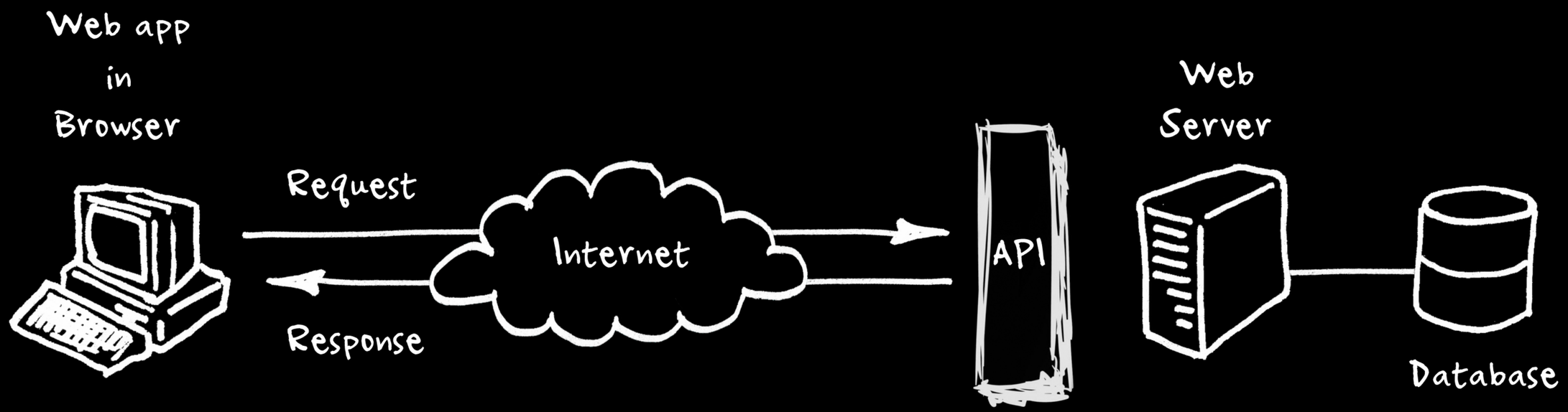
Json

Десериализация данных из API

API - **A**pplication **P**rogramming **I**nterface



Загуглить
REST API
SOAP
gRPC
Graphql

Клиент- серверная модель









Что такое API?

- **API (Application Programming Interface)** – интерфейс взаимодействия с приложением или сервисом.
- Для обращения к API чаще всего используются HTTP-запросы, соответствующие заданной спецификации.
- В ответ на апи запрос выдается текстовый ответ в формате JSON

JSON-файлы и их синтаксис

JSON (JavaScript Object Notation) – текстовый формат обмена данными, основанный на языке JavaScript. Легко читается и пишется людьми, а также анализируется и генерируется программами.

```
[
  {
    "id": 1,
    "lastName": "Иванов",
    "firstName": "Иван",
    "patronymic": "Иванович",
    "position": "Руководитель отдела",
    "email": "ivanov@company.com",
    "departmentId": 1
  },
  {
    "id": 2,
    "lastName": "Петрова",
    "firstName": "Мария",
    "patronymic": "Сергеевна",
    "position": "Менеджер",
    "email": "petrova@company.com",
    "departmentId": 2
  },
```

JSON-файлы и их синтаксис

Анкета сотрудника
Андрей начал заполнять резюме для устройства на работу и заполнил только данные о своем имени и возрасте (26 лет).

Устройство на полный рабочий день – да
Адрес – Санкт-Петербург, Невский проспект, 28
Языки программирования – Python, C#, C++

example.json

```json
{
    "name": "Andrew",
    "age": 26,
    "isFullTime": true,
    "address":
    {
        "city": "Санкт-Петербург",
        "street": "Невский проспект",
        "house": 28
    },
    "progLanguages": ["Python", "C#", "C++"]
}
```

Форматы данных в API (JSON)

JSON *(JavaScript Object Notation)*

- **Набор пар ключ-значение (как в словаре)**
- **Порядок пар не важен, может меняться от запроса к запросу**
- **Ключ — всегда строка**
- Значение:
- строка
- число
- null
- Логический
- true или false
- массив (перечисление через запятую)
- объект (другой JSON)

```
{
    "name": "Иван Иванович",
    "age": 24,
    "isMarried": false,
    "children": null,
    "cars": ["а123бв10",
"б321ав01"],
    "mobilePhone": {
        "name": "Samsung",
        "made": 2015
    }
}
```

Оформление Json

Плохо:

```
{
   user_name: "Mohammad Faisal",
   user_id: "1"
}
```

Хорошо:

```
{
   userName: "Mohammad Faisal",
   userId: "1"
}
```

Используйте camelCase для JSON свойств

Статус-коды:

1xx — информационные

2xx — успешное выполнение

3xx — перенаправление

4xx — ошибка на стороне клиента

5xx — ошибка на стороне сервера

4xx иногда заменяют на 2xx



Статус-коды:

200 – OK – успешный запрос. Если клиентом были запрошены какие-либо данные, то они находятся в заголовке и/или теле сообщения.
201 – OK – в результате успешного выполнения запроса был создан новый ресурс.
204 – OK – ресурс успешно удалён.
304 – Not Modified – клиент может использовать данные из кэша.
400 – Bad Request – запрос невалидный или не может быть обработан.
401 – Unauthorized – запрос требует аутентификации пользователя.
403 – Forbidden – сервер понял запрос, но отказывается его обработать или доступ запрещён.
404 – Not found – ресурс не найден.
500 – Internal Server Error – разработчики API должны стараться избежать таких ошибок.

Статус-коды:

Уровень 100: информационный

100 Continue – сервер получил заголовки запроса, и клиент теперь должен передать тело запроса.

Статус-коды:

Уровень 200: успешный

200 OK – запрос был успешным.
201 Created – запрос создал новый ресурс.
202 Accepted – запрос был принят, но обработка еще не завершена.
204 No Content – запрос был обработан и вернул пустой ответ. .

Статус-коды:

Уровень 300: перенаправление

301 Moved Permanently – ресурс был перемещен в место, указанное в заголовке ответа.

Статус-коды:

Уровень 400: клиентская ошибка

400 Bad Request – запрос имеет неверный формат или невалиден.
401 Unauthorized – у клиента нет нужной авторизации.
403 Forbidden – у клиента нет прав для доступа к ресурсу.
404 Not Found – запрошенный ресурс не найден.
405 Method Not Allowed – неверный HTTP-глагол.
409 Conflict – запрос вызывает конфликт ресурсов друг с другом.

Статус-коды:

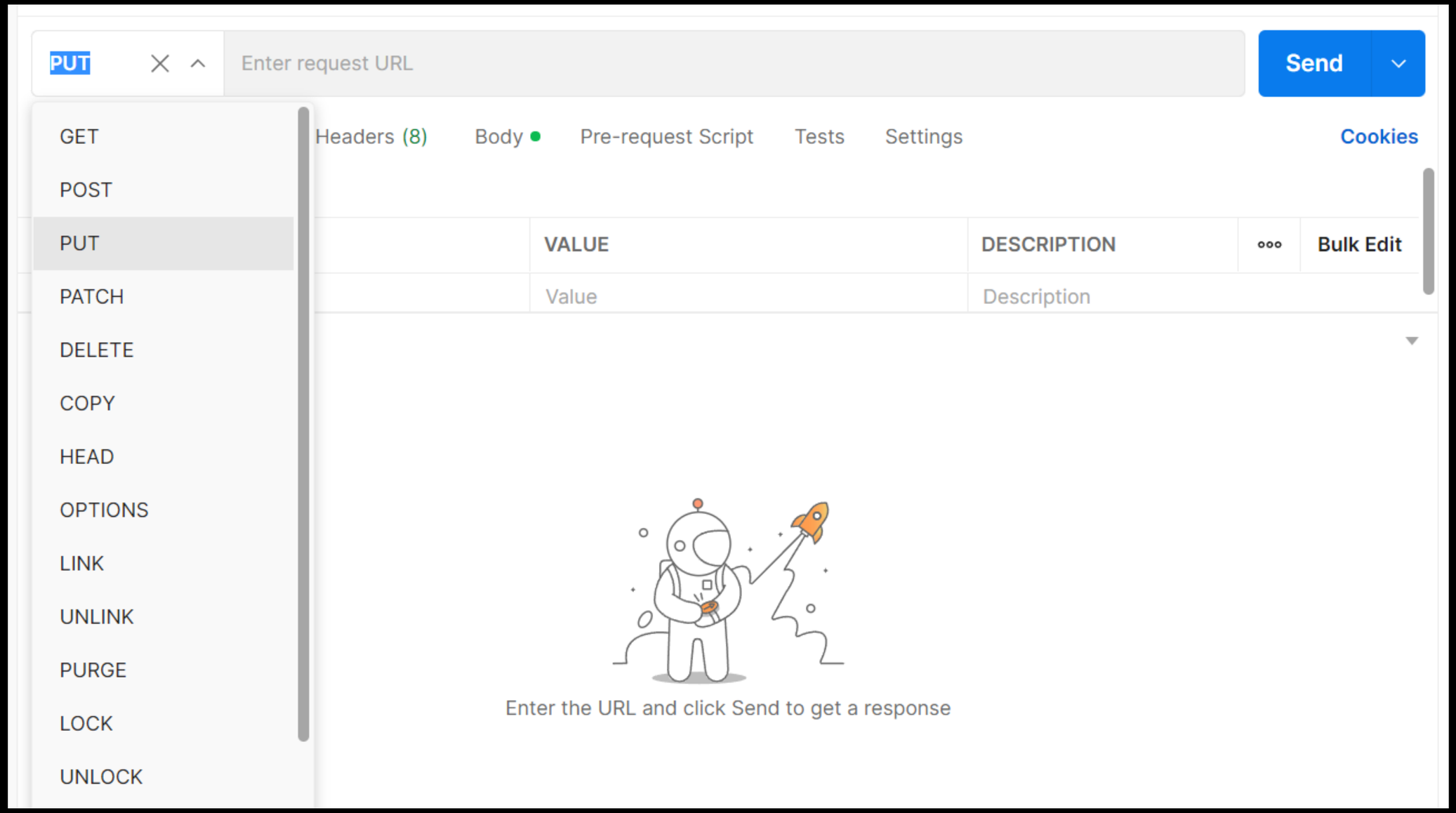
Уровень 500: серверная ошибка

500 Internal Server Error – возникло неожиданное состояние.
502 Bad Gateway – сервер получил плохой ответ от вышестоящего сервера.
503 Service Unavailable – сервер недоступен или перегружен.
504 Gateway Timeout – сервер не получил своевременный ответ от вышестоящего сервера.

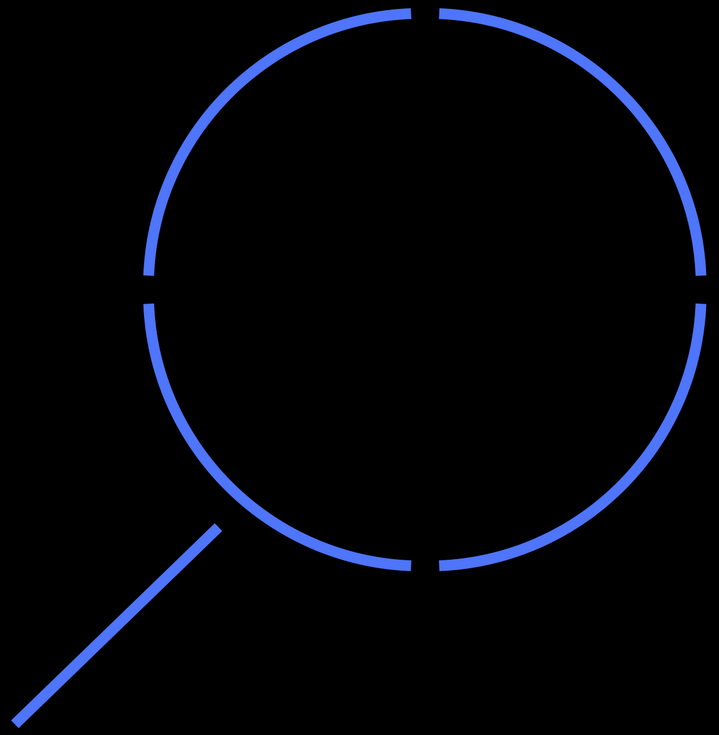
Как обратиться к API?

- Браузер
- python скрипт
- клиенты типа Swagger и Postman
- Утилиты curl и wget —инструменты командной строки, используемые для выполнения HTTP-запросов и загрузки файлов из интернета

Методы REST API

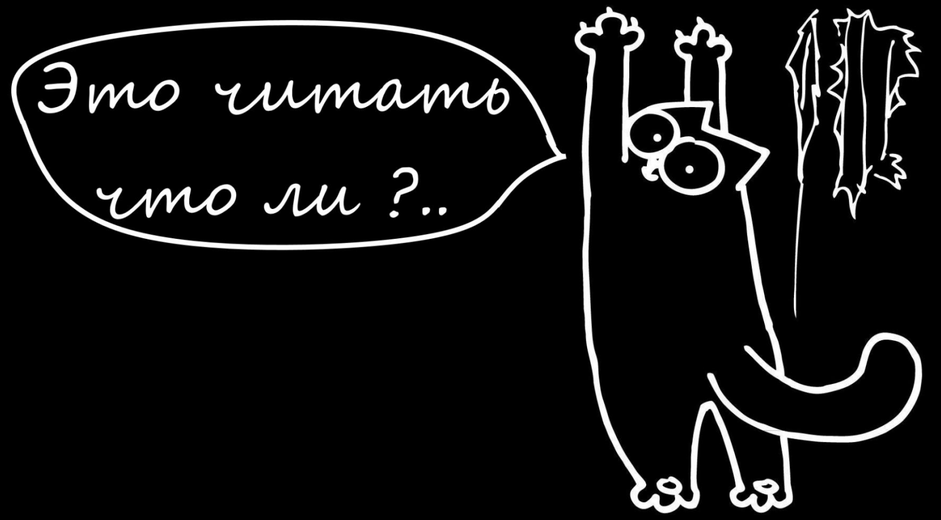


Документация для API



Зачем нужна?

Где взять?



Из чего состоит HTTP-запрос

Запрос

```
GET /wiki/страница HTTP/1.1
Host: ru.wikipedia.org
User-Agent: Mozilla/5.0 (X11; U; Linux i686; ru; rv:1.9b5) Gecko/2008050509
Firefox/3.0b5
Accept: text/html
Connection: close
(пустая строка)
```

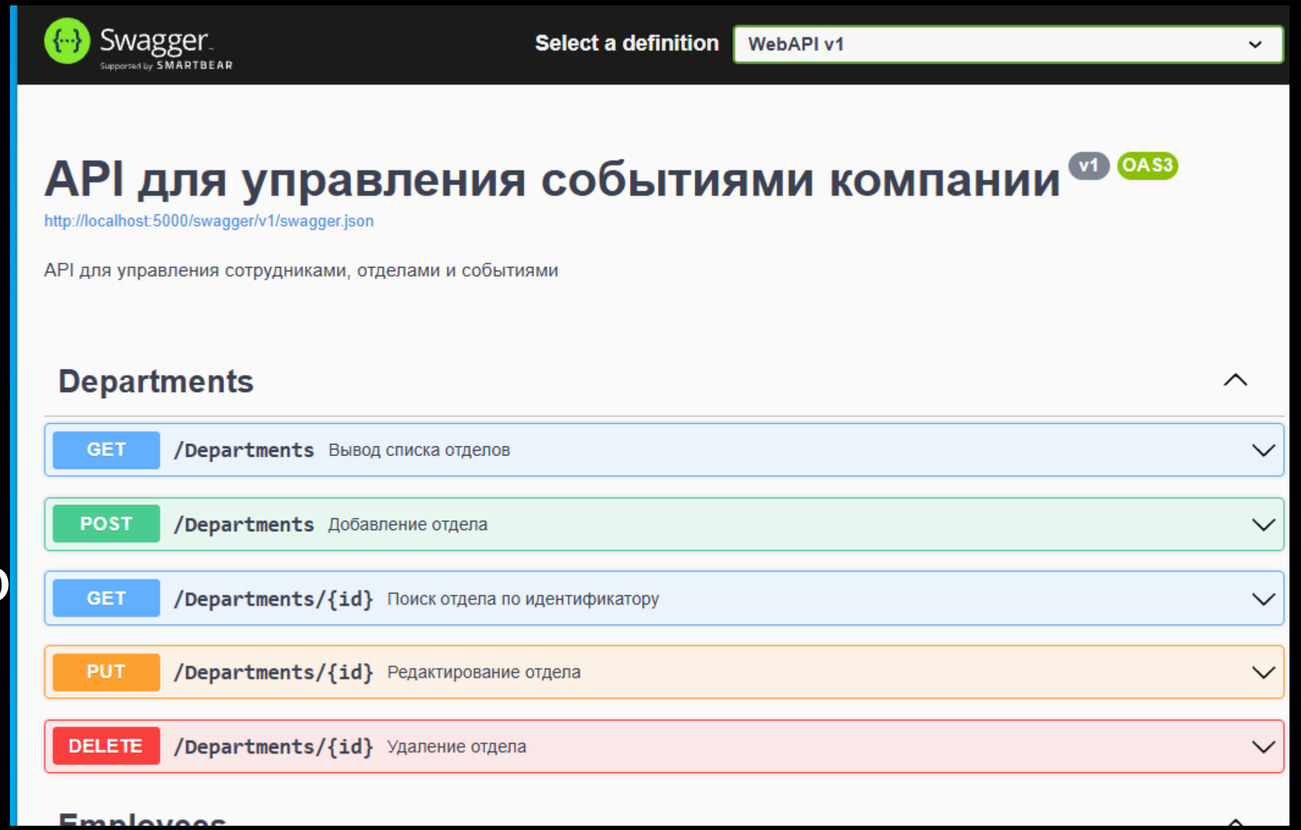
Ответ

```
HTTP/1.1 200 OK
Date: Wed, 11 Feb 2009 11:20:59 GMT
Server: Apache
X-Powered-By: PHP/5.2.4-2ubuntu5wm1
Last-Modified: Wed, 11 Feb 2009 11:20:59 GMT
Content-Language: ru
Content-Type: text/html; charset=utf-8
Content-Length: 1234
Connection: close
(пустая строка)
(далее следует запрошенная страница в HTML)
```

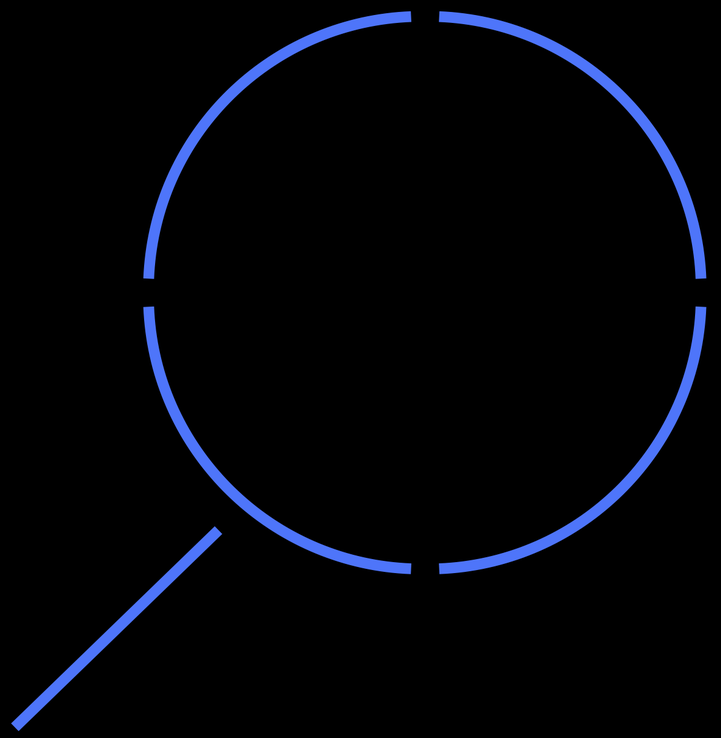
- **URL / endpoint /**
- **Метод**
- **Query-параметры**
- **Заголовки**
- **Аутентификационные данные**
- **Тело запроса**

Документирование REST API

1. Код постоянно меняется
2. Время на документацию не выделяют
3. Документация не успевает за изменениями
4. Никто не умеет писать документацию
5. Когда начинать вести документацию?
6. Не хочу писать документацию



Документация для API



- Swagger
- Apidoc:
- Текстовые документы

Примеры использования различных методов для взаимодействия с ресурсами

- *GET /books/* – получает список всех книг. Как правило, это упрощённый список, т.е. содержащий только поля идентификатора и названия объекта, без остальных данных.
- *GET /books/{id}* – получает полную информацию о книге.
- *POST /books/* – создаёт новую книгу. Данные передаются в теле запроса.
PUT /books/{id} – изменяет данные о книге с идентификатором {id}, возможно, заменяет их. Данные также передаются в теле запроса.
- *DELETE /books/{id}*– удаляет данные с идентификатором {id}.

Примеры использования различных методов для взаимодействия с ресурсами

HTTP-мето	Конечная точка API	Описание
GET	`/products`	Получить список продуктов
GET	`/products?limit=x`	Получить только x товаров (к примеру, только 5 товаров)
GET	`/products/<product_id>`	Получить один конкретный продукт
POST	`/products`	Создать новый продукт
PUT	`/products/<product_id>`	Обновить товар
PATCH	`/products/<product_id>`	Частично обновить товар
DELETE	`/products/<product_id>`	Удалить товар

URL должен отражать структуру вложенных ресурсов

- `GET /shops/2/products` получить список продуктов из магазина 2.
- `GET /shops/2/products/31` получить детали продукта 31 из магазина 2.
- `DELETE /shops/2/products/31` удалить продукт 31 из магазина 2.
- `PUT /shops/2/products/31` обновить данные о продукте 31. Используйте PUT на URL ресурса, а не коллекции.
- `POST /shops` создать новый магазин и вернуть данные о нём. Используйте POST на URL коллекции.

REST: что важно помнить

1. Работает на основании HTTP/HTTPS
2. Основной формат передачи данных —JSON (есть и другие)
3. Запрос/ответ состоит из 3 частей: Строка состояния, заголовки, тело

Совет 1.

Если вы хотите получить всех пользователей.

Плохо:

```
GET /user или GET /User
```

Хорошо:

```
GET /users
```

Используйте множественное число для коллекций

Совет 2.

Вместо этого пользуйтесь HTTP методами для описания операций.

Плохо:

```
POST /updateuser/{userId} или GET /getusers
```

Хорошо:

```
PUT /user/{userId}
```

Не используйте глаголы в URL ресурсе

Совет 3.

Плохо:

```
{
   user_name: "Mohammad Faisal",
   user_id: "1"
}
```

Хорошо:

```
{
   userName: "Mohammad Faisal",
   userId: "1"
}
```

Используйте camelCase для JSON свойств

Совет 4.

Плохо:

```
GET /shops/123?token=some_kind_of_authenticaiton_token
```

Хорошо:

Вместо этого пользуйтесь заголовками.

Не передавайте аутентификационные токены в URL

Как тестировать API?

Что проверять?

- Статус-код ответа
- Структура ответа
- Если вносились изменения, то они видны:
- в приложении
- в базе-данных
- в ответе



Как выполнить API:

- 1. Ознакомьтесь с документацией API: для каждого API обычно есть документация, которая описывает доступные методы и параметры, а также примеры запросов и ответов. Эта документация поможет вам понять, как использовать API и какие данные можно получить.
- 2. Используйте инструменты для тестирования API: существует множество инструментов для тестирования API, таких как Postman, SoapUI, Swagger и др. Они позволяют отправлять запросы к API и просматривать ответы. Использование этих инструментов поможет вам быстро протестировать API и проверить его работоспособность.
- 3. Тестирование методов API: методы API - это действия, которые вы можете выполнить с помощью API, например, получить список товаров, создать новый заказ или обновить данные пользователя. При тестировании методов API необходимо проверить, что они работают правильно, возвращают корректные данные и обрабатывают ошибки.
- 4. Проверка форматов данных: API может возвращать данные в различных форматах, таких как JSON, XML или CSV. При тестировании API необходимо проверить, что данные возвращаются в корректном формате и соответствуют ожидаемой структуре.
- 5. Обработка ошибок: при использовании API возможны ошибки, например, неверный запрос или отсутствие данных. При тестировании API необходимо проверить, что ошибки обрабатываются правильно и возвращаются корректные сообщения об ошибке.
- 6. Автоматизация тестирования API: автоматизация тестирования API позволяет быстро и эффективно тестировать API, особенно при большом количестве методов и параметров. Для автоматизации тестирования API можно использовать языки программирования, такие как Python, Java или JavaScript, или инструменты автоматизации тестирования, такие как Postman или SoapUI.

Попрактикуемся в работе с API?



Установка библиотеки

Выполните в терминале:
pip install requests

Импорт библиотеки

```
import requests
```

Отправка GET-запроса

response = requests.get('http-запрос')

Установка таймаута на запрос

```
requests.get('https://api.example.com', timeout=5)  # 5 секунд
```

Таймауты для соединения и чтения

```
requests.get('https://api.example.com', timeout=(3, 7))
# 3 сек на соединение и 7 на получение ответа
```

Проверка кода ответа

```
print(response.status_code)  # 200 означает успех
```

Получение текста ответа

```
print(response.text)
```

Вызов и подключение библиотеки requests

```
import requests
response = requests.get('https://yandex.ru/pogoda/')
print(response.text)
```

get() – функция из библиотеки **requests**, позволяющий отправлять GET-запросы к указанному серверу

text – атрибут, предоставляющий доступ к телу ответа в виде строки.

Получение данных в формате JSON

```
data = response.json()
print(data)
```

Попробуем получить содержимое страницы sports.ru

```
import requests
page = requests.get('https://sports.ru')
print(page.status_code) # 200 OK
print(page.text)  # содержимое страницы
```

Ответ

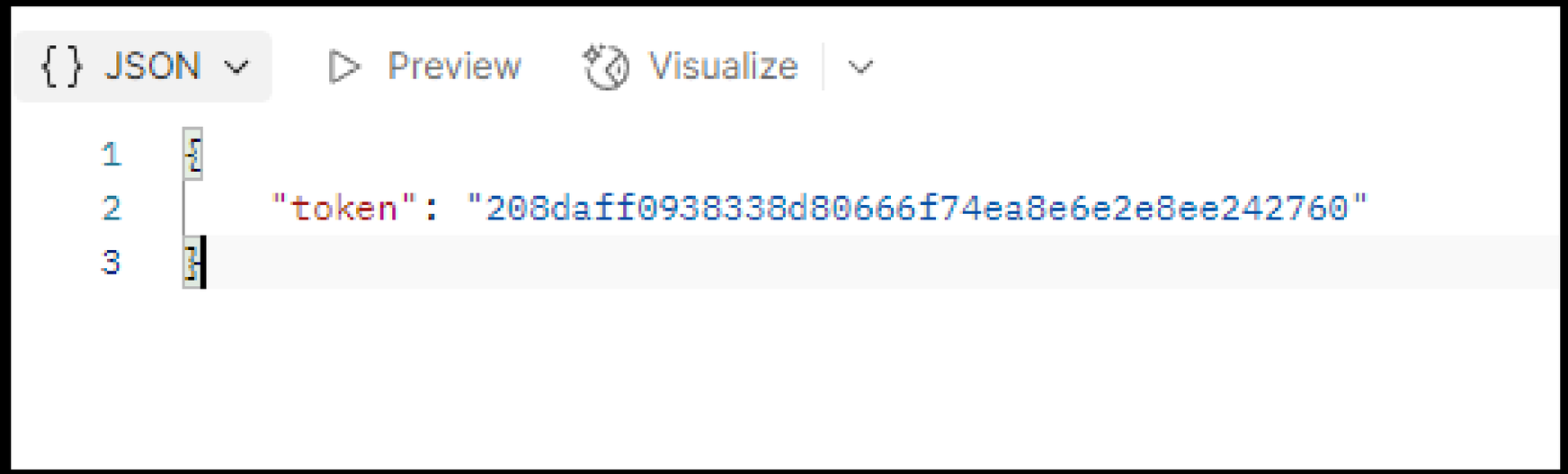
```
import requests
data = {"name": "Иван", "age": 30, "city": "Москва"}
jprint(data)

# Вывод ответа, через пользовательскую функцию jprint
print("response:\n{}\n\n".format(response))
print("response.url:\n{}\n\n".format(response.url))                #Посмотреть формат URL (с параметрами)
print("response.headers:\n{}\n\n".format(response.headers))        #Header of the request
print("response.status_code:\n{}\n\n".format(response.status_code)) #Получить код ответа
print("response.text:\n{}\n\n".format(response.text))              #Text Output
print("response.encoding:\n{}\n\n".format(response.encoding))      #кодировка используемая Requests
print("response.content:\n{}\n\n".format(response.content))        #В бинарном виде
print("response.json():\n{}\n\n".format(response.json()))         #JSON Output
```

Доступ к данным JSON

```
json_data = response.json()
token = json_data.get("token")

print(data)
```



Проверка содержимого ответа перед обработкой

```
if response.ok and 'application/json' in
response.headers.get('Content-Type', ''):
    data = response.json()
else:
    print('Ответ не содержит JSON')
```

Обработка ошибок парсинга JSON

```
try:
    data = response.json()
except ValueError:
    print('Ошибка преобразования JSON')
```

Передача параметров в GET-запросе

```
params = {'q': 'requests', 'page': 2}
response = requests.get('http-запрос', params=params)
```

Отправка POST-запроса

```
payload = {'key': 'value'}
response = requests.post('http-запрос', data=payload)
```

Отправка PATCH -запроса

```
payload = {'key': 'value'}
response = requests.patch('http-запрос', json=payload)
```

Отправка Delete -запроса

```
response = requests.delete(f"{BASE_URL}/products/21")
print(response.json())
```

Обработка ошибок запроса

```
try:
    response = requests.get('http-запрос')
    response.raise_for_status()
except requests.HTTPError as e:
    print(f'Ошибка: {e}')
```

Хранение токена в заголовке Authorization

```
import requests

headers = {'Authorization': 'Bearer abc123'}
response = requests.get('https://api.example.com/data',
headers=headers)

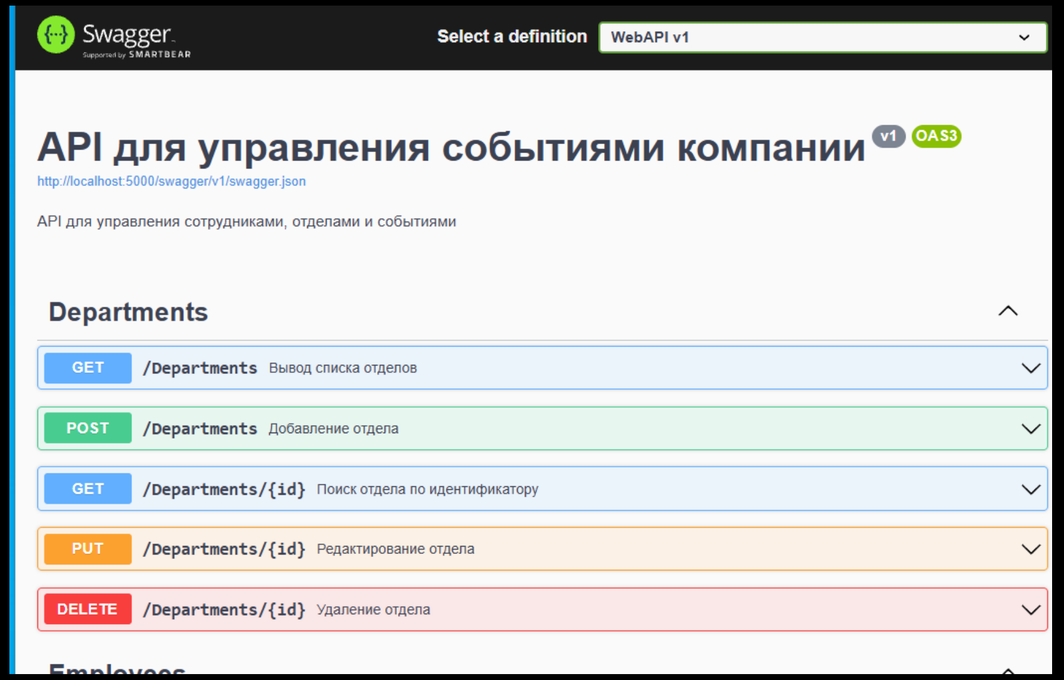
print(response.text)
```

Запросы к API

```
>>> r = requests.get('https://api.github.com/events')
>>> r = requests.post('https://httpbin.org/post', data =
{'key':'value'})
>>> r = requests.put('https://httpbin.org/put', data =
{'key':'value'})
>>> r = requests.delete('https://httpbin.org/delete')
>>> r = requests.head('https://httpbin.org/get')
>>> r = requests.options('https://httpbin.org/get')
```

Структура проекта

```
my_project/
├── app/
│   ├── main.py              # Точка входа в приложение
│   ├── api/                 # Взаимодействие с API
│   │   ├── __init__.py
│   │   ├── departments.py   # Функции для запросов к Departments
│   │   ├── employees.py     # Функции для запросов к Employees
│   │   ├── events.py        # Функции для запросов к Events
│   │   └── tags.py          # Функции для запросов к Tags
│   └── core/                # Настройки, конфигурации
│       ├── __init__.py
│       └── config.py        # Загрузка переменных окружения
├── .env                     # Переменные окружения (BASE_URL)
├── .gitignore
├── requirements.txt         # Зависимости (requests, python-dotenv)
└── README.md                # Описание проекта
```



Файл app/api/__init__.py

```
from . import departments, employees, events, tags
```

Файл .env

```
BASE_URL=http://localhost:5000
TIMEOUT=30
```

Файл app/core/config.py:

```
import os
from dotenv import load_dotenv

load_dotenv()

class Settings:
    BASE_URL: str = os.getenv("BASE_URL", "http://localhost:5000")
    TIMEOUT: int = int(os.getenv("TIMEOUT", 30))

settings = Settings()
```

Файл requirements.txt

requests>=2.31.0
python-dotenv>=1.0.0

Файл app/api/tags.py:

```
import requests
from core.config import settings

BASE_URL = settings.BASE_URL
TIMEOUT = settings.TIMEOUT


def get_tags():
    """GET /Tags - Просмотр всех хэш-тегов"""
    response = requests.get(f"{BASE_URL}/Tags", timeout=TIMEOUT)
    return response.json

def create_tag(data):
    """POST /Tags - Добавление хэш-тега"""
    response = requests.post(f"{BASE_URL}/Tags", json=data,
timeout=TIMEOUT)
    return response.json
```

```
def get_tag(tag_id):
    """GET /Tags/{id} - Поиск хэш-тега по идентификатору"""
    response = requests.get(f"{BASE_URL}/Tags/{tag_id}", timeout=TIMEOUT)
    return response.json


def update_tag(tag_id, data):
    """PUT /Tags/{id} - Редактирование хэш-тега"""
    response = requests.put(f"{BASE_URL}/Tags/{tag_id}", json=data, timeout=TIMEOUT)
    return response.json
```

```
def delete_tag(tag_id):
    """DELETE /Tags/{id} - Удаление хэш-тега"""
    response = requests.delete(f"{BASE_URL}/Tags/{tag_id}", timeout=TIMEOUT)
    return response.json
```

Файл app/main.py

```
from api import departments, employees, events, tags
from core.config import settings

def main():
    print(f"Подключение к API: {settings.BASE_URL}")

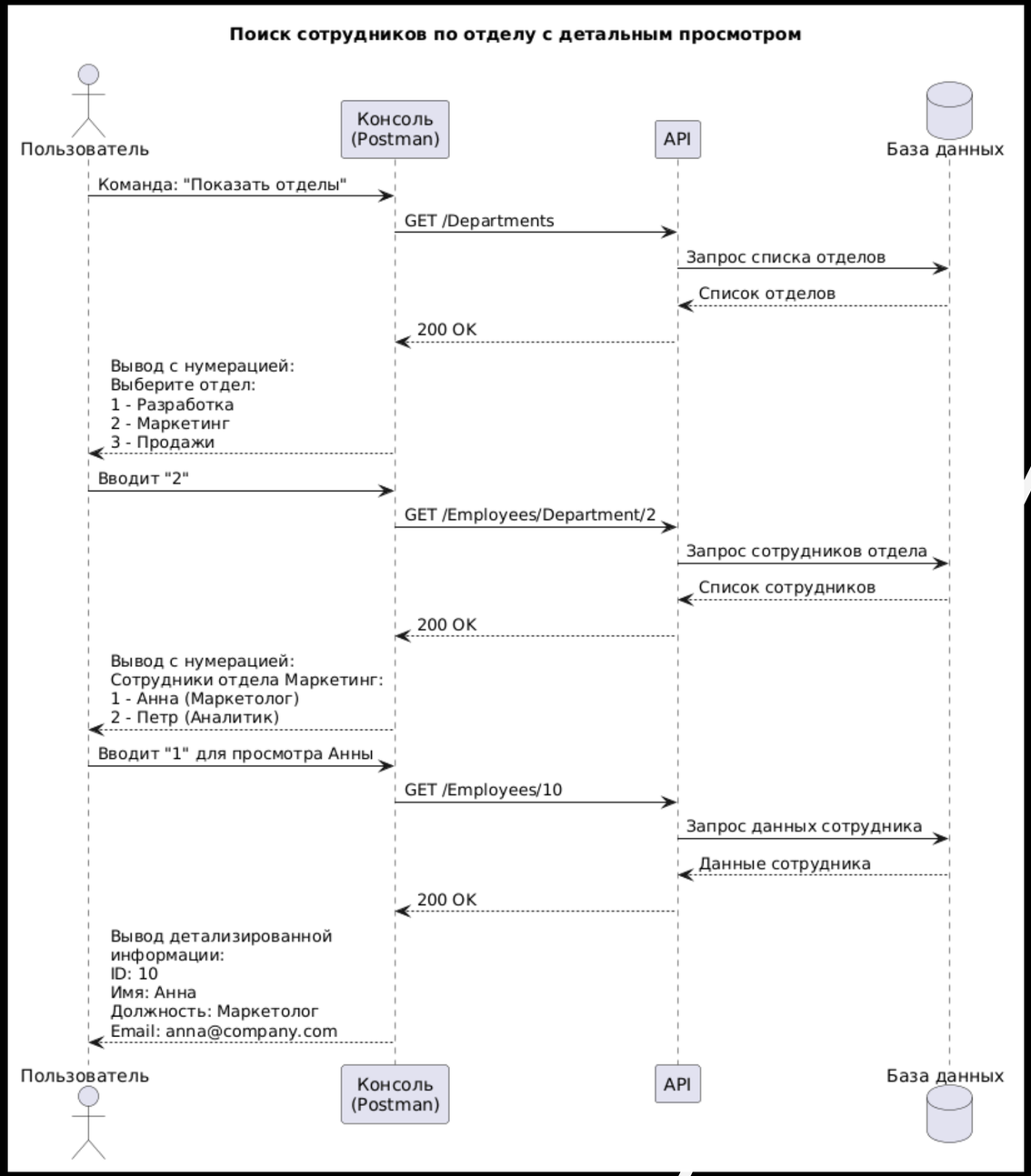
    # Пример: получить список отделов
    response = departments.get_departments()
    if response.status_code == 200:
        print("Отделы:", response.json())
    else:
        print(f"Ошибка: {response.status_code} - {response.text}")

if __name__ == "__main__":
    main()
```

ПРАКТИЧЕСКОЕ ЗАДАНИЕ

Поиск сотрудников по отделу с детальным просмотром

Цель
Разработать сценарий взаимодействия пользователя с API через консольное приложение для поиска сотрудников по номеру отдела и просмотра детальной информации о сотруднике.



ПРАКТИЧЕСКОЕ ЗАДАНИЕ

Описание процесса

1. Получение списка отделов
- Пользователь инициирует команду "Показать отделы" в консоли
- Консоль отправляет GET-запрос к эндпоинту `/Departments` для получения списка всех отделов
- Консоль получает ответ, форматирует данные и выводит пронумерованный список отделов для выбора

2. Выбор отдела пользователем
- Пользователь вводит номер отдела (в примере — "2", что соответствует отделу "Маркетинг")
- Консоль определяет ID выбранного отдела для последующего запроса

3. Получение сотрудников выбранного отдела
- Консоль отправляет GET-запрос к эндпоинту `/Employees/Department/{departmentId}` с ID выбранного отдела
- API запрашивает список сотрудников из базы данных по указанному отделу
- Консоль нумерует полученных сотрудников и выводит их пользователю

4. Просмотр детальной информации о сотруднике
- Пользователь выбирает номер сотрудника из списка (в примере — "1", сотрудник Анна)
- Консоль отправляет GET-запрос к эндпоинту `/Employees/{id}` для получения детальной информации
- API запрашивает данные конкретного сотрудника из базы данных
- API возвращает полную информацию о сотруднике (включая ID, имя, должность, email) с кодом 200 OK
- Консоль выводит детализированную информацию пользователю